Demucs: Hybrid Spectrogram and Waveform Source Separation

Deep Learning for Music Source Separation & Karaoke Application

Rustom Firdosh Bhesania

Speech Analysis Course
Institute for Natural Language Processing (IMS)
University of Stuttgart
Adviser: Dr. Wolfgang Wokurek

January 2026

Section 1

Introduction

Music Source Separation

What is MSS?

- Separating mixed audio into individual components
- Extract vocals, drums, bass, and other instruments
- Classic "cocktail party problem"
- Applications: remixing, karaoke, music analysis

Challenges

- Non-linear mixing effects (reverb, compression)
- Overlapping frequency content
- Phase information loss
- Limited training data
- Real-time processing requirements

Historical Context

Traditional Approaches (Pre-2015)

- Non-negative Matrix Factorization (NMF)
- Independent Component Analysis (ICA)
- Phase cancellation techniques
- **Problem**: Poor quality, artifacts, limited to specific scenarios

Deep Learning Era (2015-2020)

- Spectrogram-based methods (STFT + masking)
- Open-Unmix, Spleeter
- **Issue**: Phase reconstruction, high-frequency artifacts

Waveform Revolution (2019+)

- Conv-TasNet for speech separation
- **Demucs**: First successful waveform-based music separator

Demucs Architecture Evolution

Demucs v1 & v2: Waveform Domain (2019)

Key Innovation: Direct waveform processing

Architecture:

- U-Net inspired encoder-decoder
- 5 encoder + 5 decoder layers
- Bidirectional LSTM bottleneck
- GLU (Gated Linear Units) activations
- Layer normalization

Advantages:

- No phase reconstruction needed
- 10 seconds of context
- Better percussive instrument separation

Training:

- Dataset: MUSDB18 (87 songs)
- Loss: L1 time-domain loss
- Optimizer: Adam
- Batch size: 64

Results:

- SDR: ~6.3 dB average
- Beat spectrogram methods
- Human evaluation: reduced artifacts

Why Waveform Domain?

Spectrogram Limitations:

```
Time-Frequency Trade-off
|-- Short window -> Good time, poor frequency
|-- Long window -> Good frequency, poor time
+-- Phase discarded -> Information loss
```

Conv-TasNet Artifacts:

- Hollow instrument attacks
- Missing notes (see bass example)
- Constant broadband noise
- Linear encoder/decoder cannot synthesize content

Demucs Solution:

- Non-linear encoding/decoding
- Deep convolutional network
- Learn source representations in context
- Can "fill in" lost information

Demucs v3: Hybrid Domain (2021)

Architecture Innovation:

- **Two parallel U-Nets**:
  - Time domain (temporal convolutions)
  - Frequency domain (spectral via STFT)
- Representations summed after layer 5
- Shared 6th encoder/decoder layer
- Symmetric decoder with ISTFT

Why Hybrid?

- Time: Good for transients (drums)
- Frequency: Good for tonal (vocals)
- Best of both worlds

Performance:

- **Won Sony MDX Challenge 2021**
- SDR: ~7.6 dB on MUSDB HQ
- Significant reduction in bleeding
- Better across all sources

Training Data:

- MUSDB HQ + 800 extra songs
- Total: ~887 songs
- Critical for performance

Hybrid Demucs Architecture Diagram

```
                Input Waveform
                      |
    +-----------------+-----------------+
    |                 |                 |
Temporal            STFT            Spectral
Encoder                             Encoder
    |                                   |
T-Conv 1                            Z-Conv 1
T-Conv 2                            Z-Conv 2
T-Conv 3                            Z-Conv 3
T-Conv 4                            Z-Conv 4
T-Conv 5                            Z-Conv 5
    |                                   |
    +------------- Sum ----------------+
                  |
            Shared Layer 6
         (BiLSTM + Attention)
                  |
    +------------ Split ---------------+
    |                                   |
T-Conv 6                            Z-Conv 6
T-Conv 7                            Z-Conv 7
T-Conv 8                            Z-Conv 8
T-Conv 9                            Z-Conv 9
T-Conv 10                           Z-Conv 10
    |                                   |
    |            ISTFT                  |
    +-----------------+-----------------+
                      |
             Output Waveform
        (Vocals, Drums, Bass, Other)
```

Demucs v4: Hybrid Transformer (2022)

Architecture Overview

Motivation:

- Can long-range context improve separation?
- Transformers excel at long sequences
- Music has structure (verses, chorus)

Key Changes:

- Keep outermost 4 encoder/decoder layers
- Replace innermost 2 with Transformer
- More flexible than Hybrid Demucs
- No strict dimension alignment needed

Transformer Design:

- Self-attention within each domain
- Cross-attention between domains
- Layer Scale initialization
- 8 attention heads
- Feed-forward networks
- Normalization before attention

Sparse Attention:

- Extended receptive field
- Memory efficient
- State-of-the-art results

Cross-Domain Transformer Encoder

Component Structure:

1. **Transformer Encoder Layer** (per domain):
   - Layer Norm, Multi-Head Self-Attention, FFN
   - Layer Scale for stable training
   - Residual connections
2. **Cross-Attention Layer**:
   - Spectral queries attend to temporal keys/values
   - Temporal queries attend to spectral keys/values
   - Information exchange between domains
3. **Interleaved Processing**:
   - Self-Attention (Time) then Cross-Attention
   - Self-Attention (Freq) then Cross-Attention

Benefits:

- Learns which domain is best for each feature
- Dynamic integration of temporal and spectral info
- Handles heterogeneous data shapes

Performance Comparison

Model	Training Data	SDR (dB)	Notes
Demucs v1	MUSDB (87)	6.3	Waveform only
Conv-TasNet	MUSDB (87)	6.8	Artifacts present
Demucs v2	MUSDB (87)	6.5	Improved architecture
Hybrid Demucs v3	MUSDB + 800	7.6	**MDX Winner 2021**
HT Demucs v4	MUSDB + 800	9.00	Transformer base
HT Demucs v4 (FT)	MUSDB + 800	**9.20**	**State-of-the-art**

Key Insights:

- Extra training data crucial (+800 songs = +2 dB)
- Hybrid better than single domain
- Transformers improve with data
- Fine-tuning adds 0.2 dB

Human Evaluation Results

Mean Opinion Score (1-5 scale)

Quality & Artifacts:

Model	Drums	Bass	Vocals	Other
Open-Unmix	3.2	3.4	3.8	3.1
Conv-TasNet	3.5	3.6	4.0	3.3
Demucs v1	3.9	4.1	4.2	3.7
Hybrid Demucs	**4.3**	**4.4**	**4.5**	**4.1**

Contamination (Bleeding):

Model	Score
Open-Unmix	3.5
Conv-TasNet	3.7
Demucs v1	4.0
Hybrid Demucs	**4.4**

Study Details:

- 38 participants
- 20 samples each
- 8-second excerpts
- MUSDB test set

Technical Implementation Details

Training Configuration:

Hyperparameters:

- Learning rate: 3e-4
- Initial channels: 100
- Optimizer: Adam
- Batch size: 64
- Weight rescaling: 0.1
- Segment length: 10 seconds
- Overlap: 0.25

Data Augmentation:

- Random remixing
- Pitch shifting
- Time stretching
- Channel shuffling
- Source dropping

Computational Requirements:

- GPU: 7GB VRAM (recommended)
- Minimum: 3GB with –segment 8
- Processing: ~1.5x track duration
- CPU mode available

Model Variants Available

Model Name	Description	Use Case
htdemucs	HT Demucs base	Default, best balance
htdemucs_ft	Fine-tuned	4x slower, better
htdemucs_6s	6 sources	+Guitar, Piano (experimental)
hdemucs_mmi	Hybrid v3	Faster, good quality
mdx	MDX Track A	MusDB HQ only
mdx_extra	MDX Track B	Extra training data

Karaoke Mode: `--two-stems vocals` (separates vocals from accompaniment)

Key Research Contributions

Novel Ideas in Demucs

1. **Waveform-Domain Separation for Music**
   - First successful time-domain separation
   - Avoids phase reconstruction artifacts
   - Enables synthesis of missing information
2. **Hybrid Architecture**
   - Parallel processing of time and frequency
   - Complementary strengths
   - Automatic domain selection per feature
3. **Cross-Domain Transformers**
   - Self-attention within domains
   - Cross-attention across domains
   - Flexible for heterogeneous data
4. **Training Insights**
   - Importance of extra training data
   - L1 loss effectiveness
   - Per-source fine-tuning benefits

Limitations & Future Work

Current Limitations:

- Data hungry: Need 800+ extra songs
- Memory intensive: Attention requires VRAM
- 6-source model: Piano has artifacts
- Processing time: 4x slower fine-tuned
- No real-time: ~1.5x track duration

Future Directions:

- More efficient attention mechanisms
- Better performance with limited data
- Improved piano separation
- Real-time processing capabilities
- Extension to more sources
- Better handling of processed music

Karaoke Application

Project Overview

System Architecture:

```
User Input (YouTube URL)
        |
Backend (Flask + Python)
        |
1. Download Audio (yt-dlp)
        |
2. Vocal Separation (Demucs)
        |
3. Lyrics Fetch (syncedlyrics)
        |
Frontend (React)
        |
Karaoke Playback
```

Features Implemented:

- YouTube audio download
- Demucs vocal separation
- Dual-track playback
- Independent volume control
- Time-synced lyrics display
- Lyrics sync (in progress)
- Modern UI with gradients

Backend Architecture

Technology Stack:

- **Flask**: Web server framework
- **yt-dlp**: YouTube audio extraction
- **Demucs**: Vocal/instrumental separation
- **syncedlyrics**: LRC format lyrics
- **FFmpeg**: Audio format conversion

Key Functions:

1. `download_audio(url)` - Downloads MP3
2. `separate_audio_demucs(path)` - Runs Demucs
3. `get_lyrics(title, artist)` - Fetches lyrics
4. `parse_lrc(content)` - Parses timestamps

API Endpoints:

- `POST /api/process` - Main pipeline
- `GET /api/audio/<track>/<file>` - Serves audio
- `POST /api/cleanup` - Removes temp files

Demucs Integration Details

Separation Command:

```
cmd = [
    'demucs',
    '--two-stems', 'vocals',  # Karaoke mode
    '--out', output_dir,
    '--mp3',                  # Output format
    '--mp3-bitrate', '320',   # High quality
    audio_path
]
```

Output Structure:

```
output_dir/
+-- htdemucs/
    +-- songname/
        |-- vocals.mp3
        +-- no_vocals.mp3
```

Lyrics Synchronization Challenge

Current Status: In Progress

Approach:

1. Fetch LRC format via syncedlyrics
2. Parse timestamp format
3. Convert to seconds
4. Match current playback time
5. Display current and next lines

LRC Format Example:

```
[00:12.00]First line of lyrics
[00:15.50]Second line here
[00:18.75]Third line follows
```

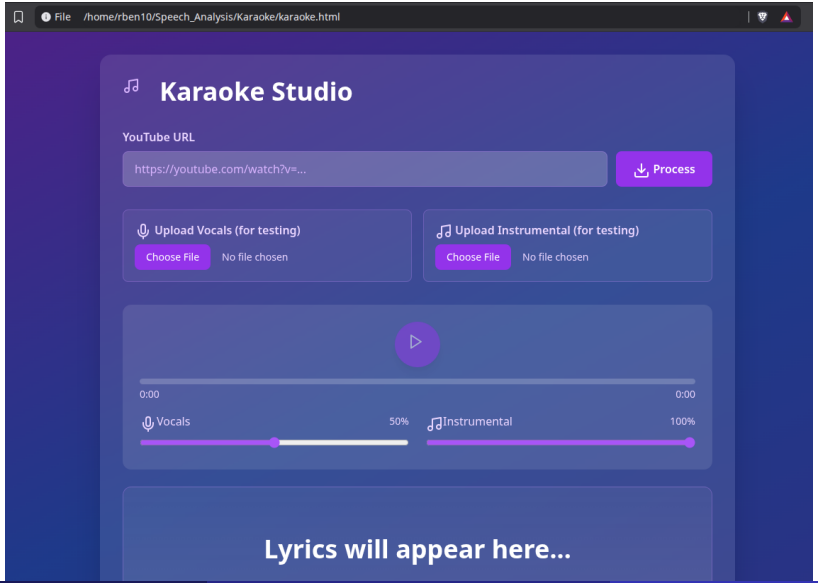
Challenges:

- Lyrics not perfectly synced
- Timestamp accuracy varies
- Need offset adjustment
- Some songs lack synced lyrics
- Fallback to unsynced needed

Planned Improvements:

- Manual offset adjustment slider
- Auto-sync using vocal analysis
- Fallback to genius.com
- User-editable timestamps
- Line-by-line highlighting

Application Screenshots



Technical Challenges Overcome

1. **Audio Synchronization**:
   - Problem: Tracks drift apart
   - Solution: Use timeupdate from one source
2. **Demucs Output Path**:
   - Problem: Dynamic folder structure
   - Solution: Glob pattern matching
3. **CORS Issues**:
   - Problem: Frontend can't access API
   - Solution: Flask-CORS configuration
4. **Memory Management**:
   - Problem: Large files consume memory
   - Solution: Cleanup endpoint
5. **File Format Compatibility**:
   - Problem: Various download formats
   - Solution: Force MP3 conversion

Performance Metrics

Processing Pipeline Timings:

Step	Duration
YouTube Download	~5-30s
Demucs Separation	~4-8 min
Lyrics Fetch	~1-2s
Total	**~5-10 min**

For 3-4 minute song, hardware dependent

Hardware Requirements:

- **CPU Mode**: Slow (1.5x min)
- **GPU Mode**: RTX 3060+
- **RAM**: 8GB min, 16GB rec.
- **Storage**: ~100MB per song

Optimization Ideas:

- Pre-cache popular songs
- Use smaller Demucs model
- Implement job queue
- Add progress websocket

Demo Workflow

Live Demonstration:

1. **Select a Song**:
   - Copy YouTube URL
   - Example: Popular karaoke track
2. **Process Audio**:
   - Paste URL and click "Process"
   - Watch status updates
   - Wait for separation (~5-8 min)
3. **Karaoke Session**:
   - Adjust vocal volume to 0%
   - Keep instrumental at 100%
   - Follow lyrics on screen
   - Sing along!
4. **Custom Controls**:
   - Increase vocal to hear original
   - Seek through song
   - Test manual file upload

Future Enhancements

Short Term (2-4 weeks):

1. Complete lyrics synchronization
2. Manual offset adjustment
3. Lyrics highlighting
4. Song history/playlist
5. Configurable Demucs models

Medium Term (1-2 months):

1. Real-time progress (WebSocket)
2. Waveform visualization
3. Pitch detection & scoring

Medium Term (continued):

4. Local file upload support
5. Multi-language lyrics

Long Term (3+ months):

1. Multi-user karaoke sessions
2. Video synchronization
3. Advanced mixing (EQ, reverb)
4. Leaderboard & scoring
5. Mobile app version

Learning Outcomes

From Demucs Paper:

- Music source separation techniques
- Waveform vs. spectrogram trade-offs
- Importance of hybrid architectures

From Implementation:

- React state management for audio
- Web Audio API and sync playback
- Full-stack application architecture

Speech Analysis Connections:

- Source separation for speech enhancement
- Similar techniques for speaker diarization
- Time-frequency analysis concepts
- Deep learning architectures for audio

Conclusion

Summary

Demucs Achievements:

- Revolutionary waveform separation
- Hybrid architecture combining domains
- Cross-domain Transformers
- State-of-the-art 9.20 dB SDR
- Won multiple competitions
- Open-source and accessible

Application Success:

- Functional karaoke system
- Demucs integration working
- Clean, modern interface
- Dual-track playback
- Lyrics display (sync in progress)
- Demonstrates practical MSS

Impact: Demucs democratized music source separation, enabling karaoke, remixing, and music education.

References

Key Papers:

1. Defossez, A. (2019). "Music Source Separation in the Waveform Domain"
2. Defossez, A. (2021). "Hybrid Spectrogram and Waveform Source Separation"
3. Rouard, S., Massa, F., Defossez, A. (2022). "Hybrid Transformers for Music Source Separation"

Code & Resources:

- GitHub: facebookresearch/demucs
- Dataset: MUSDB18-HQ
- Tools: yt-dlp, Flask, React, syncedlyrics

Related Work:

- Conv-TasNet (Luo & Mesgarani, 2018)
- Open-Unmix (Stoter et al., 2019)
- Wave-U-Net (Stoller et al., 2018)
- Spleeter (Deezer, 2019)

Thank You!

Questions?
Email: rustom.bhesania10@gmail.com
GitHub:
github.com/rustombhesania/Speech_Analysis_karaoke

Acknowledgments:

- Dr. Wolfgang Wokurek (Adviser)
- Facebook AI Research (Demucs)
- MUSDB dataset creators
- Open-source community
- IMS, University of Stuttgart

Appendix

A: Demucs Model Comparison

Feature	v1/v2	v3	v4
Domain	Time	Time+Freq	Time+Freq+Trans
Encoder	5	5+5	4+4+Trans
Bottleneck	BiLSTM	BiLSTM	Cross-Domain Trans
Context	10s	10s	Longer (sparse)
SDR	6.3	7.6	9.20
Data	MUSDB	MUSDB+800	MUSDB+800
Params	~33M	~83M	~89M

B: Application Architecture Diagram

Data Flow Overview:

1. **Frontend (React)** - User submits YouTube URL
2. **Backend (Flask)** - Processes request
   - Downloads audio with yt-dlp
   - Separates vocals with Demucs
   - Fetches lyrics with syncedlyrics
3. **Response** - JSON with audio URLs and lyrics
4. **Playback** - React loads and syncs audio tracks

API Structure:

- Request: `POST /api/process` with `{url: "youtube.com/..."}`
- Response: `{vocal_url, instrumental_url, lyrics: [{time, text}]}`
- Audio serving: `GET /api/audio/<track>/<file>`

C: Code Highlights - Backend

Demucs Separation Function:

```
def separate_audio_demucs(audio_path):
    output_dir = tempfile.mkdtemp(dir=OUTPUT_FOLDER)

    cmd = ['demucs', '--two-stems', 'vocals',
           '--out', output_dir, '--mp3',
           '--mp3-bitrate', '320', audio_path]

    subprocess.run(cmd, check=True,
                   capture_output=True)

    base_name = os.path.splitext(
        os.path.basename(audio_path))[0]

    separated_dir = os.path.join(
        output_dir, 'htdemucs', base_name)

    return (
        os.path.join(separated_dir, 'vocals.mp3'),
        os.path.join(separated_dir, 'no_vocals.mp3')
```

D: Code Highlights - Frontend (1/2)

Synchronized Playback:

```
const togglePlayback = () => {
  if (!vocalAudioRef.current ||
      !instrumentalAudioRef.current) return;

  if (isPlaying) {
    vocalAudioRef.current.pause();
    instrumentalAudioRef.current.pause();
  } else {
    vocalAudioRef.current.play();
    instrumentalAudioRef.current.play();
  }
  setIsPlaying(!isPlaying);
};
```

Time Synchronization:

```
const handleTimeUpdate = () => {
  if (vocalAudioRef.current) {
    setCurrentTime(vocalAudioRef.current.currentTime);
  }
```

D: Code Highlights - Frontend (2/2)

Volume Control:

```
useEffect(() => {
  if (vocalAudioRef.current) {
    vocalAudioRef.current.volume = vocalVolume;
  }
  if (instrumentalAudioRef.current) {
    instrumentalAudioRef.current.volume =
      instrumentalVolume;
  }
}, [vocalVolume, instrumentalVolume]);
```

E: Code Highlights - Lyrics (1/2)

Current & Next Line Logic:

```
const getCurrentLyric = () => {
  if (!audioData?.lyrics ||
      audioData.lyrics.length === 0)
    return '';

  const current = audioData.lyrics.filter(
    l => l.time <= currentTime
  );

  return current.length > 0 ?
    current[current.length - 1].text : '';
};
```

E: Code Highlights - Lyrics (2/2)

Next Line Logic:

```
const getNextLyric = () => {
  if (!audioData?.lyrics ||
      audioData.lyrics.length === 0)
    return '';

  const next = audioData.lyrics.find(
    l => l.time > currentTime
  );

  return next ? next.text : '';
};
```

Display Component:

```
<div className="lyrics-display">
  <p className="text-4xl font-bold">
    {getCurrentLyric() ||
     'Lyrics will appear here...'}
  </p>
  {getNextLyric() && (
```

F: Installation & Setup (1/2)

Backend Requirements:

```
# Create virtual environment
python -m venv venv
source venv/bin/activate  # Linux/Mac
# venv\Scripts\activate   # Windows

# Install packages
pip install flask flask-cors yt-dlp
pip install demucs syncedlyrics torch
```

System Dependencies:

- **FFmpeg**: Required for audio processing
  - Ubuntu: `sudo apt install ffmpeg`
  - Mac: `brew install ffmpeg`
  - Windows: Download from ffmpeg.org

F: Installation & Setup (2/2)

Frontend Setup:

- No build step required
- Open `karaoke.html` directly in browser
- Ensure backend is running first

Running the Application:

```
# Terminal 1: Start backend
python karaoke_backend.py

# Terminal 2: Open frontend
# Navigate to karaoke.html in browser
# or use: python -m http.server 8000
```

Testing:

- Backend health check: `http://localhost:5000/health`
- Expected response: `{"status": "ok"}`